

Agentic Engineering — A 5-Paper Reader's Guide

Self-evolving agents, verifiable training, and the observability layer that makes them trustworthy

From the Agentic Engineering Papers digest · 2026-05-27 · compiled
2026-05-30

May 2026

Chapter 0: The Scaffolding Is the Product

Five papers landed in the same week, aimed at five different problems — teaching agents to write their own skills, training them to drive phones, grading their reasoning, giving them memory that lasts, and catching them when they lie mid-task. Read together, they keep arriving at the same conviction from different doors: in agentic engineering, the leverage has moved off the model and into the machinery around it. The backbone is increasingly a fixed component you don't touch. The work — and the differentiation — is in the harness, the environment, the memory module, and the observability layer. This short chapter names the threads the five share so you can carry them into the chapters that follow, where each gets worked out in detail.

Thread one: the model stays frozen; you engineer everything else. This is the most literal commonality, and it's striking how consistent it is. SAM trains a small standalone memory model and bolts it onto multiple reasoning backbones without retraining any of them. MUSE-Autoskill never fine-

tunes anything — it externalizes new capability into folders of instructions, scripts, and tests that any agent can read. MobileGym leaves the agent alone and pours its engineering into a verifiable environment to train *in*. Agentic CLEAR and the trajectory-auditing paper add nothing to the model at all; they build the layer that watches it. The mental shift worth internalizing before you read on: stop asking “which model” as if that’s the lever. The papers that moved the numbers this week moved them by building better scaffolding around models they treated as given.

Thread two: capability is becoming a portable artifact, not a trained-in behavior. MUSE’s headline result isn’t its leaderboard score — it’s that a skill the agent wrote for itself, dropped unmodified into a *different* agent, lifted that agent by more than ten points. The knowledge lived in a file, not in weights, so it transferred. SAM makes the same move with memory: one trained 9B module generalizes across backbones because it’s modeled as a reusable tool, not baked into one model. Both papers are quietly arguing that the unit of reuse in agent systems should be a documented, testable component you build once and share — across agents, architectures, and teams — with no retraining and no coupling to a runtime. If that holds, the asset you’re accumulating isn’t a fine-tune; it’s a library.

Thread three: don’t trust an LLM grading a final answer — ground the verdict in something you control. This is the bundle’s sharpest practical warning, and three papers hit it from different angles. MobileGym measured it bluntly: a vision-language model judging phone trajectories from screenshots was wrong 10.2% of the time — a corrupted reward signal poisoning every RL step on top of it — so they replaced it with a deterministic diff against structured state that can’t be fooled. The trajectory-auditing paper found the same fragility in observability: an LLM judge missed four out of five logical hallucinations, while a cheap, observable “was the reasoning grounded?” flag beat every trained classifier at predicting failure. Even Agentic CLEAR, which *does* lean on LLM judges, warns that swapping the judge model reshapes your entire error profile. The portable lesson: wherever your evaluation or reward flows through a model grading prose or pixels, you’re likely carrying a double-digit error rate you can’t see. Anchor it — in a state diff, in execution-quality signals, in a typed answer form — in something deterministic.

Thread four: the final answer is the wrong unit of evaluation. Agentic CLEAR and “Beyond Final Answers” are the explicit case — both argue that pass/fail on the output throws away exactly the information you need to fix a system, and both grade the trajectory instead: where did it break, which component owned the failure, what type of error was it. But the theme runs underneath the others too. MUSE gates a new skill on its unit tests before trusting it, not on whether one task happened to pass. MobileGym’s judge flags any state change *outside* the expected outcome, catching the agent that books your ticket and also fires off a message it was never asked to send — a success the final-answer score would happily wave through. Across the bundle, “it returned the right answer” is treated as necessary and nowhere near sufficient.

Carry those four threads in as you read. The chapters are ordered to build on each other — MUSE makes capabilities, MobileGym makes a place to train them, CLEAR grades the reasoning, SAM keeps the reasoning coherent over the long haul, and the auditing paper watches the whole thing for lies — but each also stands alone, and each closes with a deliberately low-budget version of its idea

you can wire into an existing stack this week without reproducing the full apparatus. The connective tissue is the conviction that building agents you can put in production is, more and more, the work of building the system that watches them think.

Chapter 1: MUSE-Autoskill — Agents That Build Their Own Skills

Paper: [MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation](#) · arXiv 2605.27366

Here’s the practical reframing worth stealing before anything else: stop treating the procedures your agent figures out as throwaway reasoning, and start treating them as **files** — versioned, tested, reusable folders that outlive the task that produced them. That’s the whole bet of MUSE-Autoskill, and the numbers say the bet pays. When the agent distilled a skill from its own successful run and reused it, accuracy on those tasks jumped to **87.94%** — past the ceiling set by skills that human experts had hand-authored. The skill it wrote for itself beat the one a person wrote for it.

Skills as a lifecycle, not a one-shot

Most “agents that write their own tools” work stops at generation: the model produces a snippet, uses it once, and forgets it. MUSE-Autoskill (the name unpacks to Memory-Utilizing Skill Evolution) argues that a skill should move through five stages — **creation, memory, management, evaluation, and refinement** — and that skipping any of them is where existing systems leak value.

Concretely, a skill is a directory in the format Anthropic standardized for Agent Skills: a `SKILL.md` that declares the interface (name, description, inputs, outputs), plus optional `scripts/` for executable code, `resources/` for data, and `tests/` for validation. Nothing about the skill lives in opaque model weights. It’s all readable text and runnable code on disk.

The agent runs a normal plan/act/observe loop, but with two built-in skills to bootstrap everything else: `skill_create` and `web_search`. When the existing catalog can’t cover a step, the agent writes a spec, the system generates the `SKILL.md` and the supporting files, and then — this is the part that matters — the new skill is gated. The system runs the bundled unit tests inside a sandbox and **only registers the skill if the tests pass**. Fail, and the agent reads the trace, calls `update_skill` to patch the package, and re-runs. Create, evaluate, refine, register. Only validated skills enter the bank.

The two pieces engineers should actually copy

Two design choices carry most of the weight, and both are portable to whatever agent stack you’re running.

The first is **skill-level memory**. Each skill in the bank carries its own `.memory.md` file. As the agent uses the skill across different tasks, it appends notes — known failure modes, input quirks,

performance caveats (“timeout on PDFs > 100MB”). Next time the skill loads, that accumulated experience surfaces alongside the interface. This sits underneath the more familiar short-term (current task, adaptively compressed) and long-term (cross-session lessons like “prefer batched I/O”) memory layers. The novelty is binding experience to the skill, not just the session, so a procedure gets smarter every time it’s invoked rather than starting cold.

The second is **unit-test-driven evaluation as the gate**. Self-Debug and similar work already closed the loop on code generation with test traces; MUSE makes that loop the admission criterion for the entire reusable-capability bank. A skill doesn’t get to be reused until it has proven it runs. This is the difference between an agent that accumulates a pile of plausible-looking scripts and one that accumulates a library you’d trust.

Worth a nod for anyone fighting context limits: skills are managed by progressive disclosure (only the catalog of names and descriptions goes into the prompt, not the full bodies), and the agent keeps its trajectory as a DAG of turns with two-level adaptive compression — rewrite an oversized turn in place first, collapse a span of turns only if that’s not enough, and always keep the original turns recoverable in full history. It’s a clean pattern for long-horizon runs even if you never touch the skill machinery.

Why the result is more than a leaderboard win

On SkillsBench — 51 real tasks graded by automated verifiers in Docker — MUSE-Autoskill led three GPT-5.5-backed agents both with and without skills, hitting 68.40% with human skills. But the interesting number isn’t the leaderboard; it’s the transfer. The skills MUSE generated, dropped unmodified into a different agent (Hermes), lifted *that* agent by **+10.51 points**, closing 79% of its gap to human-authored skills. When both agents used the same generated skills, they landed within two points of each other. The knowledge wasn’t tied to MUSE’s internal behavior — it was genuinely externalized, a document and a script that any agent could read.

That’s the “so what.” If the capability your agent learns is a portable file rather than a trained-in behavior, you can build it once and share it across agents, architectures, and teams. No retraining, no coupling to a specific runtime.

And the economics work. Generating a skill costs roughly 383K tokens and 164 seconds — paid once. Using it is then *cheaper* than working without it: MUSE dropped about 20% in tokens and 37% in latency per task, Hermes nearly halved its tokens. The richer procedure replaces a long, noisy reasoning trajectory with a tight one, so the agent finishes in fewer turns. Generation breaks even after roughly three reuses, and the latency saving covers the creation cost on the very first reuse.

The honest caveat the paper doesn’t hide: a few skills regressed. One HVAC-control skill fell from 80% to 20% because it baked in a calibration routine tuned to one run’s noise profile — a procedure that worked once but was less robust than plain trial-and-error. The lesson for anyone adopting this: distilled skills can smuggle in source-trajectory-specific assumptions, so an audit step and the test

gate aren't optional niceties. The bottleneck on overall score was coverage, not skill quality — if the agent can't solve a task at all once, it has no trajectory to distill.

What to try, even at small scale: take a procedure your agent rediscovers on every run, freeze it into a `SKILL.md` plus a `tests/` folder, gate it on those tests, and attach a memory file it appends to over time. You don't need MUSE's full stack to get the compounding benefit — the format is just folders and files.

If MUSE-Autoskill is about agents that manufacture their own capabilities, the next chapter asks where agents *practice*. MobileGym turns its attention to mobile GUI agents and the problem of training them at all: a verifiable, highly-parallel browser-based simulation platform built so you can spin up thousands of reproducible mobile environments, reward them automatically, and benchmark them without a farm of physical phones.

Chapter 2: MobileGym — A Verifiable Gym for Mobile GUI Agents

Paper: [MobileGym: A Verifiable and Highly Parallel Simulation Platform for Mobile GUI Agent Research](#) · arXiv 2605.26114 · **Code:** github.com/Purewhiter/mobilegym

If you are training or benchmarking a phone-driving agent, the most expensive lie you are telling yourself is that a vision-language model can grade its own work. MobileGym’s headline finding is blunt about it: when the authors had a VLM judge real-device trajectories that they could also score deterministically, the VLM got the verdict wrong **10.2%** of the time — and a second judge, GPT-5.4, missed roughly the same share, just on a different set of episodes. Misjudgment that big is not a rounding error. It is a corrupted reward signal that quietly poisons every RL step you take on top of it. The fix the paper offers is almost boringly practical: stop guessing at outcomes from screenshots, and start checking them against structured state you actually control.

The trick: simulate the interaction surface, not the backend

The insight that makes everything else possible is a refusal to do too much. A GUI agent only ever sees screenshots and only ever taps, types, and swipes. So MobileGym doesn’t reproduce a real app’s servers or Android’s pixel-level internals — it reproduces the **interaction surface**: the screens, the keyboard, notifications, permission dialogs, cross-app handoffs, and the task-relevant state changes underneath them. Everything — app data, OS settings, device context — lives as **structured JSON**. App views are composed from read-mostly “world data,” a small mutable “runtime overlay” the agent writes to, and the OS runtime, then rendered for the agent through Playwright with coordinates normalized to a 0–1000 grid.

Because the whole environment is JSON, it becomes readable (you can inspect a balance or an order, which `adb` and accessibility trees can’t reliably do), writable (reset to a known start state), and forkable (clone an exact initial state *N* times). That last property is the one that matters for modern RL: group methods like GRPO need multiple rollouts from *identical* starting conditions, and live apps simply can’t give you that. A snapshot here is a JSON blob restored in milliseconds.

A judge that can’t be wrong

With full state in hand, grading stops being a similarity heuristic and becomes a diff. Each task ships a deterministic **state-based judge** that inspects the environment before and after and confirms the exact mutations it expects — `tickets[0].status: "created"`, the right train number, the right passenger. Across **416 parameterized task templates** spanning 28 apps (12 everyday, 16 system), the authors report zero false accepts and zero false rejects against the gold judges. Two design

choices earn that. First, an **AnswerSheet** protocol for query tasks: instead of matching free text — which rejects “34°C” against “34 degrees Celsius” and accepts answers that merely leak the gold number inside the agent’s reasoning — the agent fills a typed GUI form, and each field is checked by an exact-text, numeric-tolerance, or choice matcher. Second, **full-environment state comparison**, which flags any change *outside* the task’s expected outcome as an unexpected side effect. That catches the failure mode every personal-agent builder fears: the agent books your ticket and also fires off a message it was never asked to send. A screenshot judge can’t see that; a state diff can’t miss it.

Cheap enough to actually run

None of this would matter if it were as heavy as an emulator. Each browser instance is about **400 MB of RAM** with a **~3-second cold start**, against roughly 4.5–6 GB and ~78 seconds for AVD-based setups. One server held **256 parallel instances** at under 10% CPU and ~100 GB RAM, finishing a full 256-task benchmark sweep in about six minutes. For contrast, the paper cites a prior emulator-based RL setup that needed ten bare-metal servers — 960 vCPUs, 3,840 GB of RAM — to reach 512 parallel instances. MobileGym puts online RL on a single node with three GPUs, no cluster required.

Does training in a simulator survive contact with a real phone?

This is the question that sinks most simulators, so the authors tested it directly. They fine-tuned **Qwen3-VL-4B** with GRPO for ten steps on the 160-task train set. In simulation, overall success rose from **9.4% to 22.2% (+12.8 points)**, with the largest gains in the middle difficulty band — training helps most where the base model is already half-competent, and barely moves the hardest tasks, which stay capacity-limited. Then they took a 59-task subset that could be run identically on a real Redmi phone. The simulation gain there was +42.8 points; on the actual device it was **+40.7 points** — **95.1% of the training gain retained**, with sim-to-real gaps of under 4 points for both base and trained models. Since the real-device tasks used different contacts, entities, and app data, that transfer reads more like a learned policy than memorization. A small concrete win tells the story: on a Reddit post that required selecting a flair before the Post button unlocked, the base model looped on the dead button until its step budget ran out, while the trained model spotted the missing flair and finished in 22 steps.

What to take from it

The portable lesson isn’t “use MobileGym,” though if you work in mobile agents it’s now the obvious online-RL substrate to reach for. The lesson is that **verifiability is an architecture decision you make up front, not a judge you bolt on later**. If your evaluation or reward signal flows through a

VLM grading screenshots, you are very likely carrying a double-digit error rate you can't see — and the paper gives you a clean number to fear. The move that fixes it generalizes well beyond phones: represent the slice of state your task actually touches in a structured form you can snapshot, diff, and reset, then judge against the diff instead of the rendering. Even if you can't fully simulate your target environment, you can often build a thin controllable mock of the interaction surface — enough to fork identical start states for GRPO and to assert on outcomes deterministically. And borrow the AnswerSheet idea wholesale: when an agent has to *report* an answer, make it submit typed fields, not prose, so your grader checks values instead of guessing at strings. The side-effect check is worth stealing too — a passing task that quietly mutates unrelated state is a bug your final-answer score will never surface.

That last point — that a single success/failure verdict hides far more than it reveals — is exactly where the next chapter picks up. Agentic CLEAR argues that scoring an agent only on its final answer throws away most of what you'd want to know, and proposes an automated framework for grading agents at multiple levels of their reasoning and execution, so that a “right” answer reached the wrong way doesn't get a free pass.

Chapter 3: Agentic CLEAR — Evaluating Agents at Every Level

Paper: [Agentic CLEAR: Automating Multi-Level Evaluation of LLM Agents](#) · arXiv 2605.22608 ·

Code: github.com/IBM/CLEAR

If you run an agent in production, you already know the trap: the final answer looks right, the task “passes,” and yet something in the middle went sideways — a tool called four times when once would do, a planner that ordered subtasks backwards, a recovery path that never fired. Pass/fail scoring sees none of it. You catch it only by scrolling through trace logs by hand, one at a time, hoping the systemic problem jumps out before you run out of patience.

Agentic CLEAR, from IBM Research, is built to do that scrolling for you — and to do it at three zoom levels at once. The pragmatic takeaway is simple enough to act on this week: stop scoring your agent on outcomes alone, and start generating *textual diagnostics* of its behavior at the step, trace, and system levels. The tool is a `pip install clear-eval` away, runs from a single CLI command configured by a YAML file, and ships with an interactive UI for the deep dives.

Three levels of zoom, no taxonomy required

The framework runs in two stages over a collection of execution traces. First, an LLM judge evaluates every trace three ways. **Step-wise**, it critiques each individual LLM call — a quality score plus a natural-language note on what went wrong. **Trace-wise**, it judges the whole trajectory as a unit, weighing both the path taken and the final deliverable. And in a **rubric** pass, the judge first reads the task description, *generates* the criteria that task ought to satisfy, then checks the trace against the criteria it just wrote. Every judgment carries a short justification before its score, so the reasoning is always inspectable.

The second stage is where the name comes from. The instance-level critiques get fed to CLEAR (a clustering-and-summarizing method from the same group) to roll thousands of individual notes up into recurring patterns. Critiques are grouped by node — sub-agent or component — to surface failures local to, say, the planner versus the executor. Trace-level judgments aggregate into holistic system behaviors. Each surfaced insight links back to the specific step or trace that triggered it, so a finding is never a vague accusation; it’s a claim you can click into.

The design choice that matters most here is what’s *absent*: there’s no fixed error taxonomy. Most prior work hands the judge a predefined list of failure categories and measures how well it recovers them. Agentic CLEAR is taxonomy-free by design — it discovers the failure modes that fit *your* agent and *your* domain, dynamically, rather than forcing your traces into someone else’s twenty-category schema. In the experiments, it surfaced benchmark-specific issues — monkey-patching and broken

diffs on SWE-bench, policy violations and fabricated cost estimates on a customer-service benchmark, source-verification gaps on GAIA — without any benchmark-specific prompting.

Does the automated read hold up?

Two validations matter for trusting it. First, alignment with humans: the team mapped CLEAR’s auto-generated issues onto TRAIL, a hand-built taxonomy of error categories, and found every system-level issue mapped to at least one human category, collectively covering nearly all the relevant reasoning-and-planning categories. Better still, the generated issues were often *more* fine-grained and actionable than the human categories they mapped to — capturing specific failure patterns where the taxonomy offered only broad buckets.

Second, predictive signal: can the judge’s scores actually anticipate which traces will succeed? Across seven settings, yes — with the trace-level score the strongest single predictor of success, ahead of the step-wise average and the rubric proportion. But the honest finding is that the methods don’t hold uniformly. The rubric approach assumes the task description contains everything needed to define success, which breaks on adversarial tasks where the right move is to *refuse* the user — there, a generator that sees only the surface request happily rewards completing a request it should have declined. The step-wise method shines on modular agents and naturally phased tasks (locate, understand, fix a bug) and is weaker where steps don’t decompose cleanly.

One more practical wrinkle worth internalizing: **judge choice changes your diagnosis**. Running a smaller open model versus a frontier model over identical traces produced qualitatively different issue lists — the smaller judge wrote shorter, more generic, operationally flavored notes; the larger one produced longer, domain-specific findings about verification gaps and reasoning errors. Same agent behavior, different story depending on who’s grading.

What to try

Even if you never adopt the package, three ideas port cleanly into whatever harness you already have. Add a **node-grouped critique**: when an agent misbehaves, ask which component owned the failure, and aggregate by component instead of by run — that’s how you separate “the planner orders subtasks illogically” from “the executor forgets pagination.” Add a **self-generated rubric** step: have a judge derive the success criteria from the task before scoring, which gives you per-task checks without writing a rubric per task by hand — just remember it over-rewards completion on tasks that ought to be refused, so guard adversarial cases. And treat **judge selection as a tunable knob**, not a detail: pick the model whose specificity matches the depth of diagnosis you actually need, and know that swapping it reshapes your error profile.

The deeper shift is the one to keep: outcome scores tell you *whether* an agent failed; multi-level textual diagnostics tell you *where* and *why*. For anyone iterating on a real system, the second question is the one that closes the loop.

That loop closes only if the agent can also *remember* what it learned across a long run — and a diagnostic that flags “lost critical variables across steps” points straight at memory as the bottleneck. The next chapter takes that on directly with SAM, a state-adaptive memory architecture for long-horizon reasoning agents whose read/write policy is conditioned on the agent’s current state — deciding not just what to store, but when the moment calls for storing or retrieving it at all.

Chapter 4: SAM — State-Adaptive Memory for the Long Haul

Paper: [SAM: State-Adaptive Memory for Long-Horizon Reasoning Agent](#) · arXiv 2605.24468 · **Code:** github.com/qhj00/cabeza

If you’ve built an agent that browses, searches, and reasons over dozens of steps, you’ve felt the failure mode this paper is named after. The trajectory grows. The thoughts, tool calls, and observations pile up. And somewhere around round 40, the agent forgets the one fact it found in round 6 — the fact that would have ruled out the wrong branch it’s now chasing. The standard fixes all have the same shape: throw away old steps, fold them into a rolling summary, or retrieve snippets by similarity. SAM’s argument is that all three lose on long horizons for the same reason. They decide what to keep using a *fixed rule* — recency, compression ratio, embedding score — instead of asking what the agent actually needs *right now*.

The reframing worth stealing is this: treat long-horizon context not as a prefix you have to carry, but as a **memory space you navigate based on your current state**. The useful question isn’t “what happened most recently,” it’s “which parts of my own history are relevant to the decision I’m about to make.” That’s the whole pitch, and it turns out to be a load-bearing distinction.

Cues and pages: a two-view memory

Mechanically, SAM keeps two coupled copies of the interaction history. When the live context fills up to a token budget, SAM grabs that chunk as a **page** — a contiguous slice of reasoning, actions, and feedback, kept whole so local coherence survives. It then writes a compact **memory cue** for that page: a short note capturing what was established, what got ruled out, what’s still open, what might matter later. The raw page gets evicted from the live window and stashed in an external store; only the lightweight cue stays visible.

The critical design choice is that the cue is *not* a replacement for the page. It’s a handle. When the agent later decides it needs something from the past, it issues a recall request describing its current intent, picks a few candidate cues, and SAM goes back to the *underlying raw pages* and reconstructs an answer conditioned on that intent. So recall is never a verbatim replay and never a stale pre-baked summary — it’s a fresh, query-targeted reconstruction from preserved source material. Short-term continuity comes from the uncompressed recent context; long-term guidance comes from the cue bank; and the detailed past gets restored on demand only when asked for.

The ablation makes clear which half does the work. The team froze the page store and varied only how pages get read back. A rolling global summary, a plain recency window, and even dumping the *raw page contents* back in all trailed full SAM — recency barely beat having no memory at all.

Returning the raw pages didn’t close the gap either, which rules out “lossy compression” as the culprit. The win comes from **intent-conditioned recall**, not from consolidation being clever. Write simply; read adaptively.

Training a memory you can reuse

The slickest engineering decision is that SAM is a *standalone* module. The agent backbone stays frozen. Only a small memory model — here a 9B — gets trained, and the same trained module then drops onto multiple backbones without retraining any of them. That’s the part to internalize even if you never reproduce the training: memory is modeled as a reusable capability bolted onto a tool interface, not as something baked into one model’s weights.

Training it is harder than training a summarizer, because a memory’s value is *deferred* — a cue is only good if it preserves something that turns out to matter twenty steps later. So the recipe runs in two stages. First, supervised fine-tuning bootstraps from expert traces: frontier models act as the memory module on real queries, and only trajectories that produced correct final answers are kept, giving paired targets for both consolidation and recall. Then reinforcement learning sharpens it with **OAT-GRPO** (Oracle-Anchored Tree GRPO). Two ideas matter here. The rollout is structured as a *memory-call tree*: every time the agent recalls, the memory model branches into several samples that share the same context, each continued by the frozen reasoner until a leaf gets scored against the gold answer. Credit then flows back to each individual memory output instead of one sparse pass/fail bit smeared across the whole trajectory. And because that outcome signal is still thin, a second reward anchors each recall against a *committee of frontier models* — the union of their answers acts as an oracle proxy for “what a good recall here looks like,” densifying the signal and covering recalls the model would rarely explore on its own.

Where it pays off

Across four long-horizon benchmarks — BrowseComp, BrowseComp-ZH, WideSearch, and HLE — SAM is the strongest context-management method on every backbone tested, beating the best heuristic by a clear margin and the no-management baseline by more. The lead is widest exactly where memory pressure is highest, on the browsing benchmarks, and it *grows* as trajectories get longer: past 80 interaction rounds, where a single rolling summary starts shedding useful state, episodic recall is still pulling ahead. One trained 9B module drove the best result in every backbone—benchmark cell, while the heuristic baselines kept flipping rank depending on the task — a tell that their wins were coupling artifacts, not a real mechanism.

So what does this hand you? First, a diagnosis: if your long-horizon agent degrades as context grows, the bottleneck may not be context *length* but context *access* — and rolling summaries, the popular fix, are precisely what falls apart past long horizons. Second, a design you can approximate without the RL apparatus. You can build the cue-and-page split today: when you evict history, keep the raw chunk in a store and index it with a short, honest note about what it contained, then let the

agent recall by describing its current intent and reconstruct from the raw chunk rather than serving the note as the answer. Even a prompted version of that loop captures the core idea — separate lightweight write-time consolidation from intent-conditioned read-time reconstruction, and stop letting recency decide what your agent gets to remember. One practical knob from their analysis: page size in the 32K–64K range worked best; the largest 128K pages diluted the per-page intent signal.

A memory that knows *what* it stored is one thing. The next chapter asks a sharper question: can you trust *how* an agent got to its answer at all? It turns the audit lens away from the final output and onto the entire action trajectory — a methodology for catching trajectory-level hallucinations across the full chain of steps in multi-agent industrial workflows, where a confident, correct-looking answer can sit on top of a quietly broken path.

Chapter 5: Beyond Final Answers — Auditing the Whole Trajectory

Paper: [Beyond Final Answers: Auditing Trajectory-Level Hallucinations in Multi-Agent Industrial Workflows](#) · arXiv 2605.24219

A chiller-monitoring agent is asked whether Condenser Water Flow on Chiller 6 shows anomalies in the last week of April. It downloads the sensor history correctly. It confirms the right channel is present. It calls the anomaly-detection tool — which silently returns nothing. And then the summarization agent, undeterred by the empty result, reports back: “Yes, anomalies were detected.” The final answer is a clean, confident sentence. It is also entirely fabricated, built on a tool call that produced no output at all.

If you only grade the final answer, you would have to label that trajectory by reading tea leaves — there is no way to know the “yes” was invented. That is the gap this paper goes after, and the practical takeaway is blunt: **the final answer is the wrong unit of evaluation for an agent**. A multi-step agent can land on a correct answer through a broken process, or a wrong answer through sound reasoning that hit a bad tool. Output-only checks miss both. If you run agents in production and you need to trust the *process*, you need to audit the trajectory — the full Thought–Action–Observation trace — not just the last token.

Five ways a trajectory lies

The contribution that travels best is the taxonomy. The authors define five hallucination types as structural properties of the trace, each isolating a different kind of failure:

- **Factual** — a step asserts something the data contradicts. Catchable from one step in isolation.
- **Referential** — a step cites an entity, observation, or prior result that never appeared upstream. The agent “remembers” something that never happened; you can only catch it by looking at history.
- **Logical** — the reasoning doesn’t follow from its own premises, even when those premises are correct. A broken inference, not a broken fact.
- **Procedural** — the agent skips, reorders, or fabricates a required step, or claims a step finished that never ran. Invisible unless you know the prescribed workflow.
- **Scope** — an agent acts or speaks outside its mandate. The content may be perfectly correct, but it came from the wrong agent. This one is unique to multi-agent systems.

The chiller example is two of these at once — procedural (claiming completion with no valid tool result) and factual (asserting a specific anomaly that was never found). That overlap turns out to be the norm: across 225 expert-annotated trajectories, **nearly half exhibit more than one type**

simultaneously. Procedural-plus-factual and procedural-plus-scope are the most common pairs. A single binary “hallucinated: yes/no” label throws away exactly the diagnostic information you’d need to fix the system — factual failures point at grounding, procedural failures at control flow, scope failures at coordination.

The prevalence numbers reframe what an agent hallucination usually *is*. **Procedural hallucinations are the single largest category, 38.5% of all occurrences** — and they are precisely the ones output-only benchmarks can’t see. The common mental image of hallucination as “the model made up a fact” undercounts the real failure surface in agentic systems. Most of the damage is control-flow: skipping a diagnostic step, acting on a nonexistent tool output, declaring victory early.

Why your LLM judge isn’t enough

The natural reflex is to throw an LLM-as-a-judge at the traces. The paper tested that, and the result is a useful warning. At the binary level the judge looks strong — it agrees with humans on whether a hallucination exists about 79% of the time, with high recall. But agreement collapses on the subtle types. The judge missed roughly four out of five logical hallucinations and three out of four referential ones. Inter-rater agreement (Cohen’s κ) sits at a moderate 0.456 overall but drops to 0.18–0.21 for referential and logical types — low enough that even two qualified humans often disagree.

There’s an order to this difficulty, and it tracks how much context each type needs. Factual and procedural failures have overt surface cues and get caught. Referential and logical failures require reasoning across steps and slip through. Trained classifiers fared no better — fine-tuned BERT, an NLI model, and a Longformer over the full trace all landed below 0.70 ROC-AUC, under the zero-shot judge on F1. The honest framing the paper offers: **trajectory-level hallucination detection is not a solved problem, so don’t deploy a single detector and assume it covers you.**

The signal worth stealing

Here’s the finding to actually take to work. Alongside the heavy classifiers, the authors checked whether cheap execution-quality flags — already emitted during the agent loop — predict hallucination. They do, dramatically. A simple **clarity-and-justification** signal (did the agent produce clear, grounded reasoning?) hits AUC 0.908 as a *single* predictor, beating every trained classifier they built. Trajectories lacking grounded reasoning hallucinated 94% of the time; those with it, 9%. Task completion and result verification follow close behind. And when both clarity *and* result-verification are absent, the hallucination rate hits 97% — a candidate kill-switch condition for an orchestrator.

That inverts the expected engineering investment. You might assume real-time agent monitoring requires a trained trajectory model or token-level uncertainty estimates. The data says a handful of lightweight, observable runtime flags — is the reasoning grounded, did the task complete, was the

result verified — are *stronger* predictors than the expensive option, and they’re available inside the loop, early enough to halt a run before a bad action cascades downstream.

So if you operate multi-agent systems and want to trust the process, three things are portable from this work regardless of your domain. First, **log and audit the whole trace, typed** — adopt the five-category framing (factual, referential, logical, procedural, scope) so a flagged failure tells you *what* broke, not just *that* something did, and expect overlap. Second, **don’t trust a single judge or classifier** — they all underperform on the subtle cross-step types, so keep human review in the loop for referential and logical cases. Third, and most actionable, **wire cheap execution-quality signals into your orchestrator as runtime guardrails**. A clarity-and-justification check plus a result-verification check is a near-free monitor that, on this data, would have caught the silent-tool-failure that fooled the chiller agent. The paper’s whole arc is a shift in recommendation: away from post-hoc classification of finished outputs, toward lightweight signals watching the process as it runs — which is exactly the posture you want when an agent’s mistakes propagate into work orders on real infrastructure.

That theme closes the bundle as cleanly as it opened it. Across these five papers the throughline has been the same: the leverage in agentic engineering keeps turning out to live in the scaffolding around the model — the harness, the tools, the middleware, and now the observability layer that audits the trajectory rather than the answer. The model writes the words; the system around it decides whether you can trust them. Building agents you can put in production is, increasingly, the work of building the machinery that watches them think.